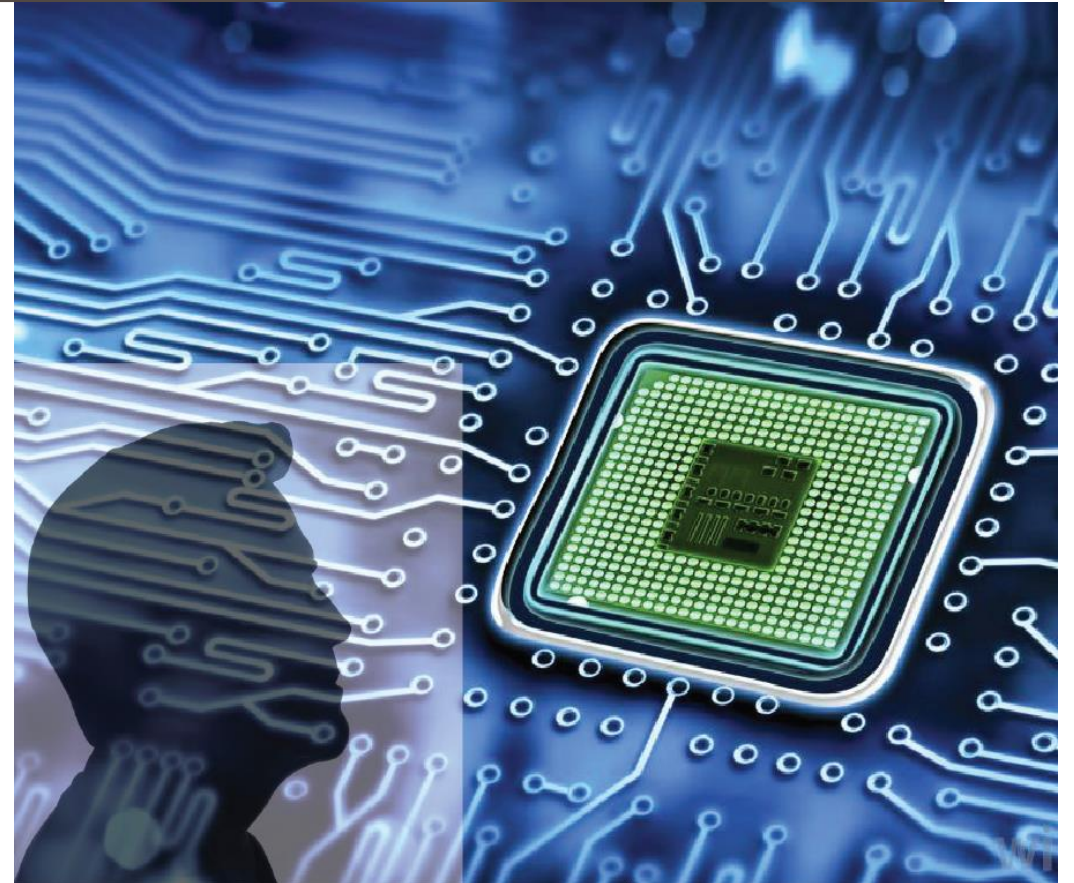


VLSI DESIGN FLOW

Week-6 Lecture-1 Logic Design

Sneh Saurabh
18th September, 2024



VDF: Revision/Announcements

-



Logic Synthesis Mechanisms



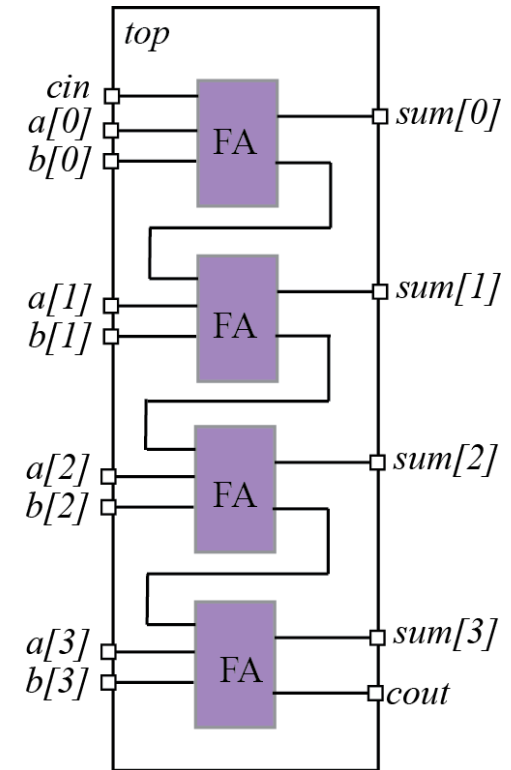
Basics

For Loop

- A for loop can model repeated logic structure
- RTL synthesis tool can unroll (expand) the loop and instantiate circuit elements for each iteration.
 - Must know the number of iterations during synthesis (not during runtime)
 - Synthesizable for a fixed number of iterations

```
module top(a, b, cin, sum, cout);  
    input [3:0]a, b; input cin;  
    output [3:0]sum; output cout;  
    reg [3:0]sum; reg cout; reg carry;  
    reg [2:0]idx;  
  
    always @(*) begin  
        carry = cin;  
        for(idx=0; idx < 4; idx=idx+1)  
            begin  
                {carry,  
                 sum[idx]}=a[idx]+b[idx]+carry  
            ;  
            end  
        cout = carry;  
    end  
  
endmodule
```

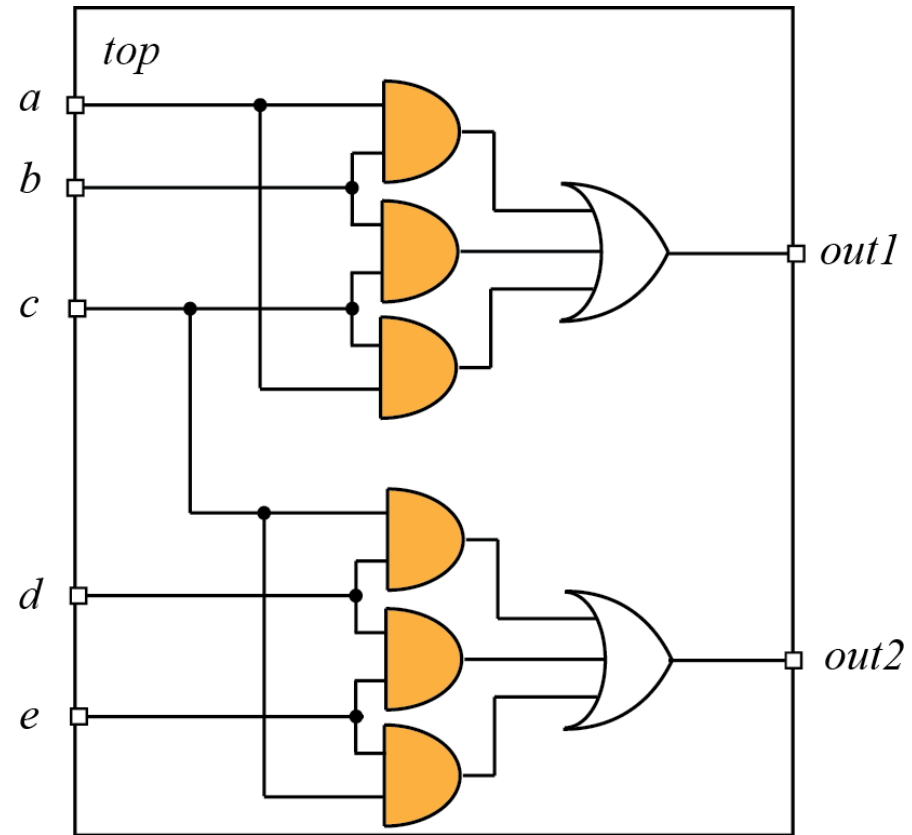
```
{carry, sum[0]}=a[0]+b[0]+cin;  
{carry, sum[1]}=a[1]+b[1]+carry;  
{carry, sum[2]}=a[2]+b[2]+carry;  
{cout, sum[3]}=a[3]+b[3]+carry;
```



Functions

Function synthesizes to a combinational logic block with one output (scalar or vector).

```
module top(a, b, c, d, e, out1, out2);  
    input a, b, c, d, e;  
    output out1, out2;  
    reg out2;  
  
    function MAJOR3;  
        input A, B, C;  
        begin  
            MAJOR3 = (A&B)|(B&C)|(C&A);  
        end  
    endfunction  
  
    assign out1 = MAJOR3(a, b, c);  
  
    always @(*) begin  
        out2 = MAJOR3(c, d, e);  
    end  
  
endmodule
```



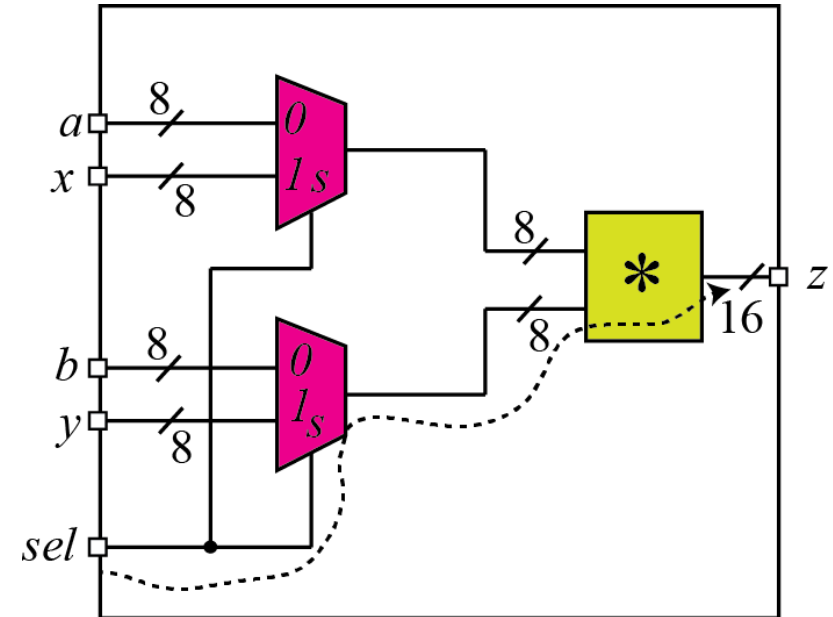
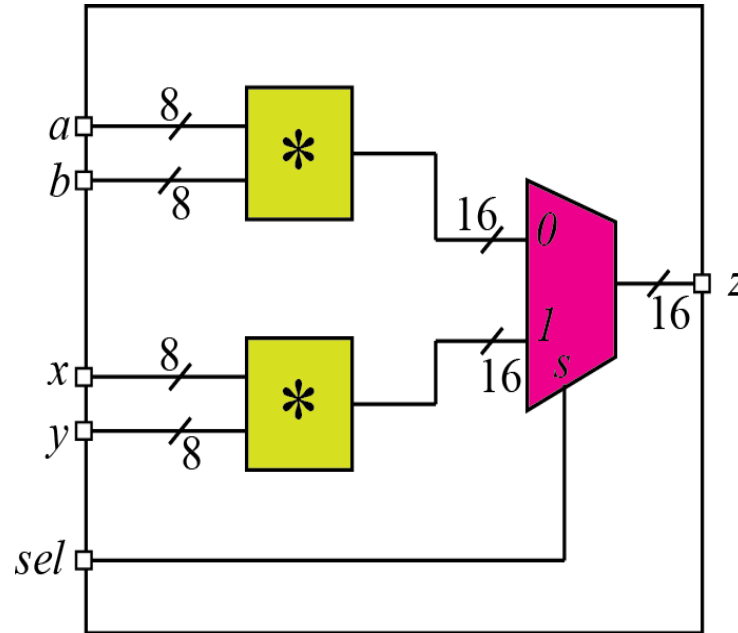
Operators: Synthesis Tasks

- Directly translate bit-wise operators and logical operators to their corresponding logic gates and optimize them subsequently.
- Arithmetic and relational operators are handled differently.
 - First translates them to some internal representation or data structure (a graph with the operator–operand relationship preserved).
 - Perform operator level optimization
 - Map to corresponding in-built implementation of operators

Operators: Resource Sharing

```
if (sel == 1'b0)
    z = a*b;
else
    z = x*y;
```

Assume: a, b, x, y are of 8 bits and z is of 16 bits.



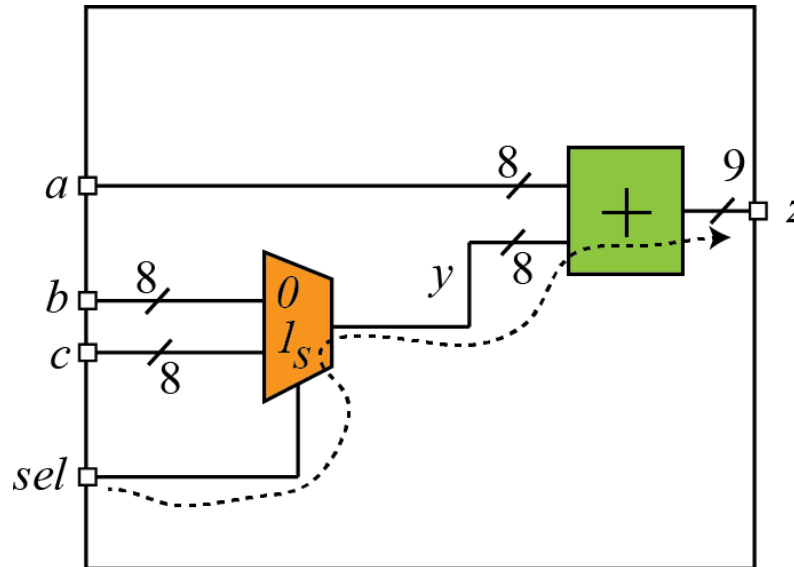
- Critical path can change
- Need to ensure that path through the select pin of the multiplexer does not violate timing constraint

Operators: Speculation (Resource Unsharing)

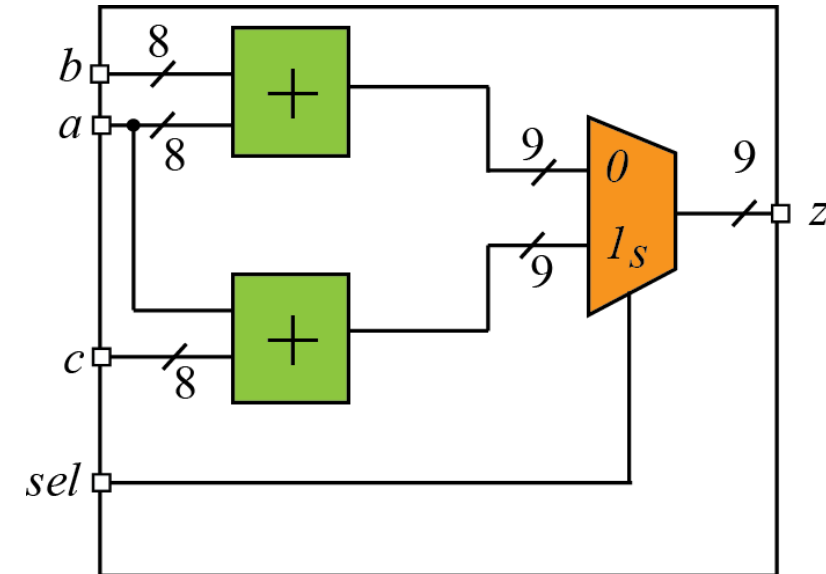
```
if (sel == 1'b0)
    y = b;
else
    y = c;

z = a+y;
```

Assume: a , b , c , y
are of 8 bits.



- Assume that path through sel is critical.



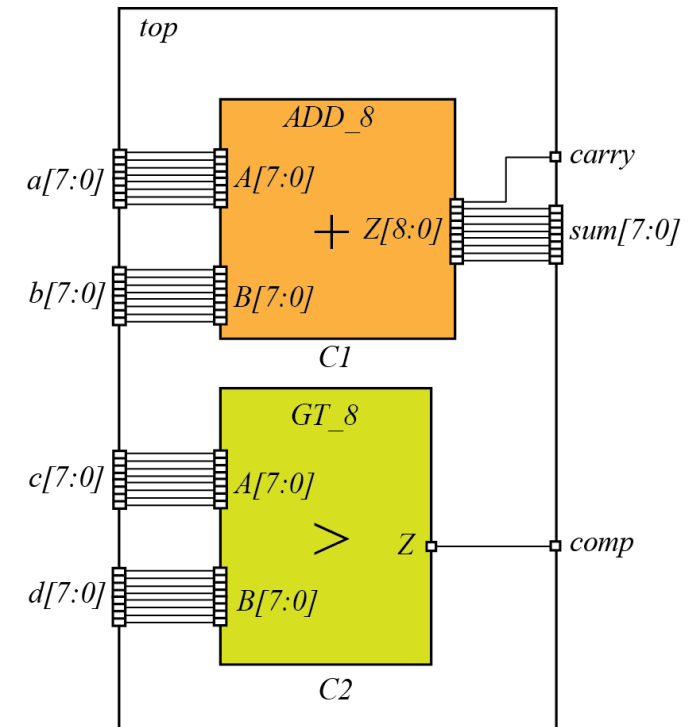
- Decreases delay of the critical path by employing more resources
- Performs addition irrespective of the sum being needed
 - speculation or eager computation

Operators: Mapping and Architecture Selection

- Tools map arithmetic operators to predefined modules implementing these operators.
- A set of internal parameterized modules implementing arithmetic operators (+, −, *, /) and relational operators (==, >, >=, <, <=).
- Instantiate these internal modules and choose the right set of parameters

```
module top(a, b, c, d, sum, carry, comp);  
    input [7:0] a, b, c, d;  
    output [7:0] sum;  
    output carry, comp;  
  
    assign {carry,sum} = a + b;  
    assign comp = (c > d);  
endmodule
```

- Exact implementation tool-specific (logical function same)
- Architecture switching in later stages possible



Formal Verification

Limitations of Simulation-based Verification

Illustration:

- Consider multiplication of two 32-bit integers
- There are $2^{32} \times 2^{32} = 2^{64}$ combinations.
- If simulation of one test vector take $1\mu\text{s}$ to simulate,
 - Will take $2^{64} \times 1 \times 10^{-6}$ seconds (≈ 0.5 million years approximately)
- Simulation-based exhaustive verification is not feasible for real-world designs

Problems of Simulation-based Verification:

- Input patterns biased towards anticipated sources of errors
 - Errors often occur where not anticipated
 - Bugs typically introduced at locations to which designers did not pay attention
- Simulation-based verification is inadequate and can never prove the correctness of a given design.
- Formal verification methodologies provide a feasible alternative.

Formal Verification as an Alternative

Formal Verification as an alternative

- Applies mathematical reasoning to establish proof of correctness
- If correctness is established, the system behaves correctly irrespective of input vectors
 - All cases implicitly covered in Formal Verification

Specification: $y = (x - 4)^2$

Design: $y = x^2 - 8x + 16$

x	Specification: $y = (x - 4)^2$	Design: $y =$ $x^2 - 8x + 16$	Pass/Fail?
0	16	16	Pass
1	9	9	Pass
2	4	4	Pass
-1	25	25	Pass
104	10000	10000	Pass

Formal verification

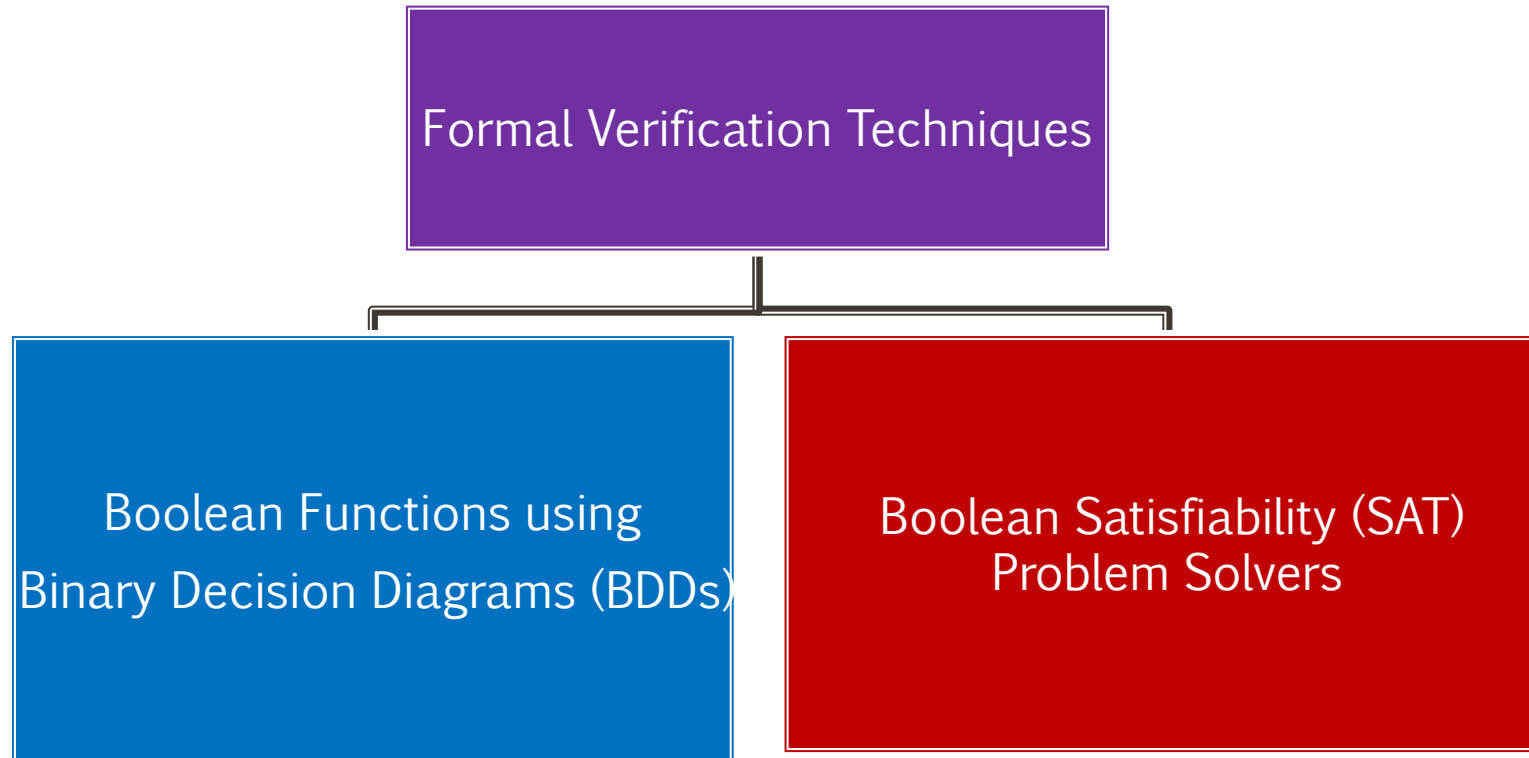
$$\begin{aligned}(x - 4)^2 &= (x - 4) \cdot (x - 4) \\ &= x \cdot (x - 4) - 4 \cdot (x - 4) \\ &= x \cdot x - x \cdot 4 - 4 \cdot x + 16 \\ &= x^2 - 8x + 16\end{aligned}$$

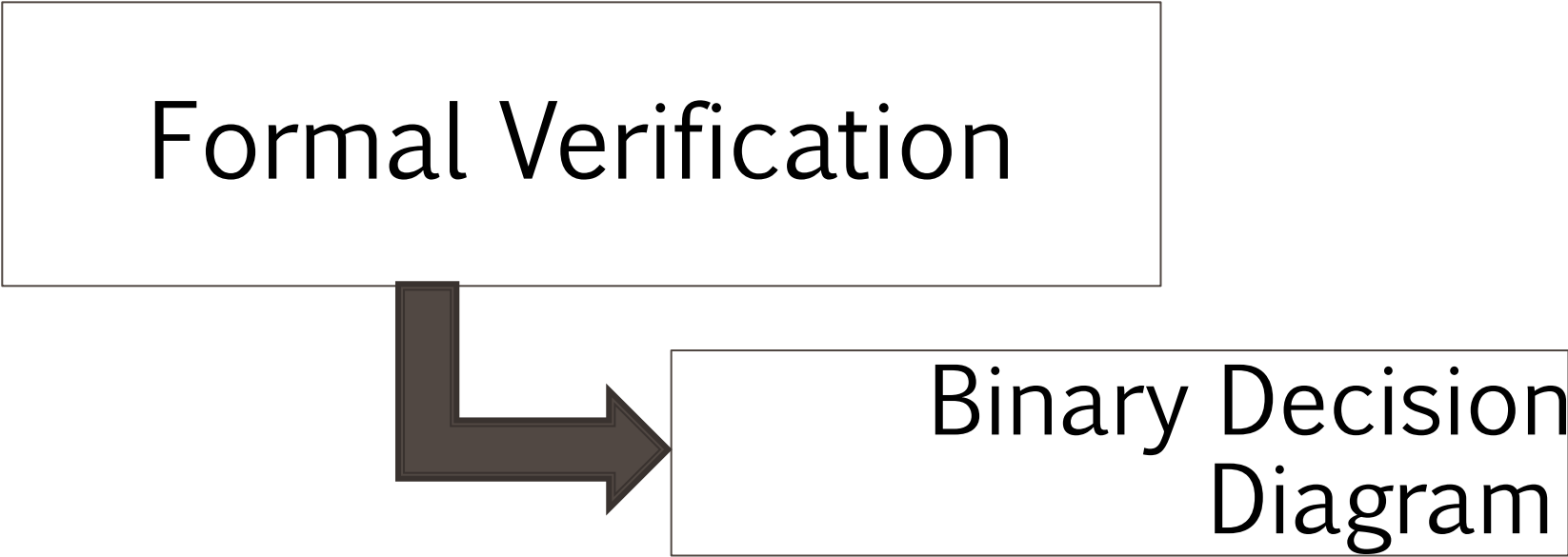
Formal Verification : Differences with Simulation

	Simulation-based verification	Formal verification
Test Vectors	Required	Not Required
Completeness	No	Yes
Mechanism	Test Vectors simulate the design and output response compared with the expected response	A mathematical proof of correctness is established or a counter-example is produced
Memory Requirement	Comparatively Low	Comparatively High

Formal Verification: Techniques

- Formal verification is computationally challenging
- Early 1990s, efficient formal verification techniques for VLSI applications developed





```
graph TD; A[Formal Verification] --> B[Binary Decision Diagram]
```

Formal Verification

Binary Decision
Diagram

Boolean Function Representations: Compactness

- Boolean functions can be represented in many ways

Compactness of Boolean function representation:

- Quantifies the growth in the size of a representation with the increase in the number of Boolean variables

- **Examples:**

- A truth table has 2^N rows for a Boolean function of N variables.
 - Truth table size increases exponentially with the number of Boolean variables
- A logic formula representation of a function: $y = ab + acd + b'd + bc'$ can be very compact

- **Desirable:** a Boolean function representation should be as small as possible.
 - It allows the corresponding data structure to consume less memory in storage and can enable easy manipulation.

Boolean Function Representations: Canonicity

Canonicity of Boolean function representation:

- If a representation is canonical, then the two equivalent functions are represented identically.
- Conversely, if a representation is canonical, and if two functions have the same representation, they are functionally equivalent.

▪ Examples:

- A truth table is a canonical representation of a Boolean function
- A logic formula is not canonical:
 - ❑ We can represent $y = ab + ac + bc$ as $y = a(b + c) + bc$, $y = ab + c(a + b)$, $y = ab + ac + b(c + aa')$, $y = ab + bc.(a + a') + ac$, etc.

- **Desirable:** In general, manipulating Boolean functions becomes easier in a canonical representation.
- For example, checking the equivalence of two functions is trivial with a canonical representation.

Compact canonical representation of Boolean functions is of great interest for logic synthesis and verification

Binary Decision Diagrams (BDD)

BDD: a data structure to represent Boolean functions canonically

- Is compact for many practically relevant functions
- Boolean operations relevant to formal verification is very efficient

- BDDs are built using *Shannon expansion*
 - Boolean Function is split into two sub-functions by assigning 0/1 to variables
 - The subfunction obtained by assigning 0 is called negative cofactor
 - The subfunction obtained by assigning 1 is called positive cofactor

$$y = ab + acd + b'd + bc'$$

$$a = 0 : y_0 = b'd + bc'$$

$$a = 1 : y_1 = b + cd + b'd + bc'$$

$$y = a'(b'd + bc') + a(b + cd + b'd + bc')$$

Binary Decision Tree

Boolean functions can be represented as **Binary Decision Tree** by applying **Shannon Expansion Theorem** recursively

$$y = x_1x_2 + x_2'x_3 + x_1'x_3'$$

Level 3: (Expand w.r.t x_3)

$$y_{x_1=0,x_2=0,x_3=0} = 1$$

$$y_{x_1=0,x_2=0,x_3=1} = 1$$

$$y_{x_1=0,x_2=1,x_3=0} = 1$$

$$y_{x_1=0,x_2=1,x_3=1} = 0$$

$$y_{x_1=1,x_2=0,x_3=0} = 0$$

$$y_{x_1=1,x_2=0,x_3=1} = 1$$

$$y_{x_1=1,x_2=1,x_3=0} = 1$$

$$y_{x_1=1,x_2=1,x_3=1} = 1$$

Level 1: (Expand w.r.t x_1)

$$y_{x_1=0} = x_2'x_3 + x_3'$$

$$y_{x_1=1} = x_2 + x_2'x_3$$

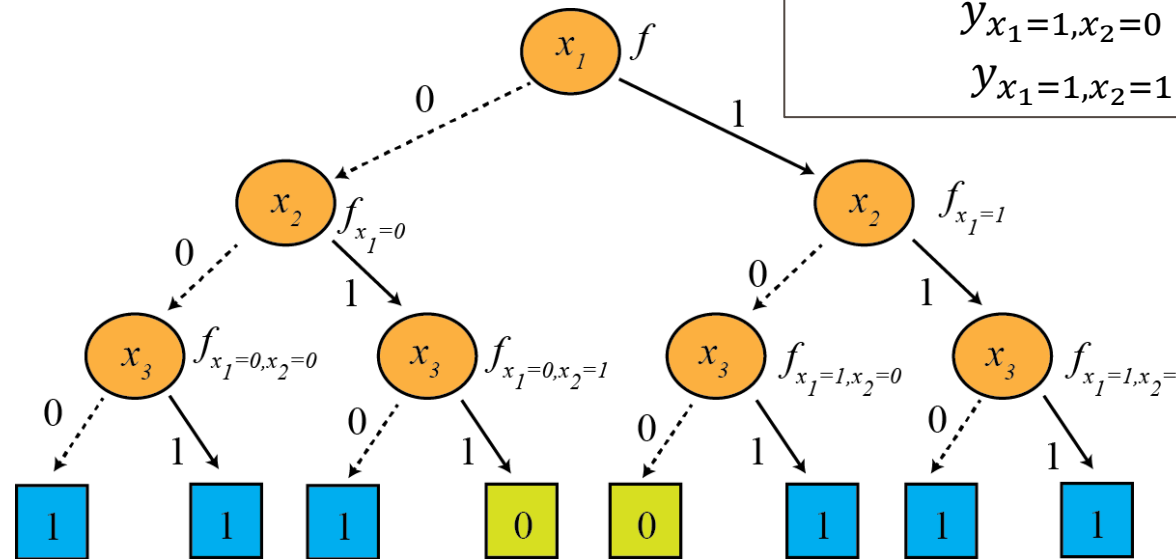
Level 2: (Expand w.r.t x_2)

$$y_{x_1=0,x_2=0} = x_3 + x_3'$$

$$y_{x_1=0,x_2=1} = x_3'$$

$$y_{x_1=1,x_2=0} = x_3$$

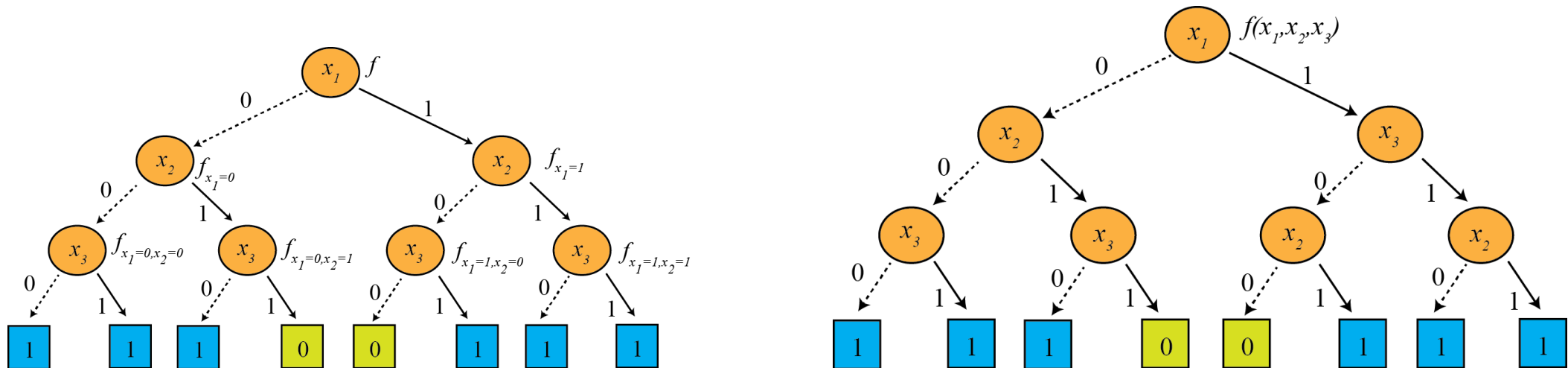
$$y_{x_1=1,x_2=1} = 1$$



To make it canonical and compact, we need to add constraint on variable ordering and remove redundancies

Binary Decision Tree: Variable Order

- At an intermediate node, we can choose any variable for expansion in a binary decision tree
- In general, different variable orders can give different binary decision tree



To enforce canonicity, we need to add constraint on the Binary Decision Tree

Ordered Binary Decision Diagram (OBDD): A BDD becomes an ordered binary decision diagram (OBDD) if the decision variables follow the same order in all the paths from the root to the leaf nodes

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.